Component-and-connect patterns:

- Broker
- Model-view Controller
- Pipe and Filter
- **Client-Server**
- Peer-to-Peer
- Service-Oriented-Architecture
- Publish-Subscribe
- Shared-Data

Client-Server Pattern

- **Context:** There are shared resources and services that large numbers of distributed clients wish to access, and for which we wish to control access or quality of service.

- **Problem:** By managing a set of shared resources and services, we can promote modifiability and reuse, by factoring out common services and having to modify these in a single location, or a small number of locations.
  - We want to improve scalability and availability by centralizing the control of these resources and services, while distributing the resources themselves across multiple physical servers.

- **Solution:** Clients interact by requesting services of servers, which provide a set of services.
  - Some components may act as both clients and servers.
  - There may be one central server or multiple distributed ones.

Client-Server Example

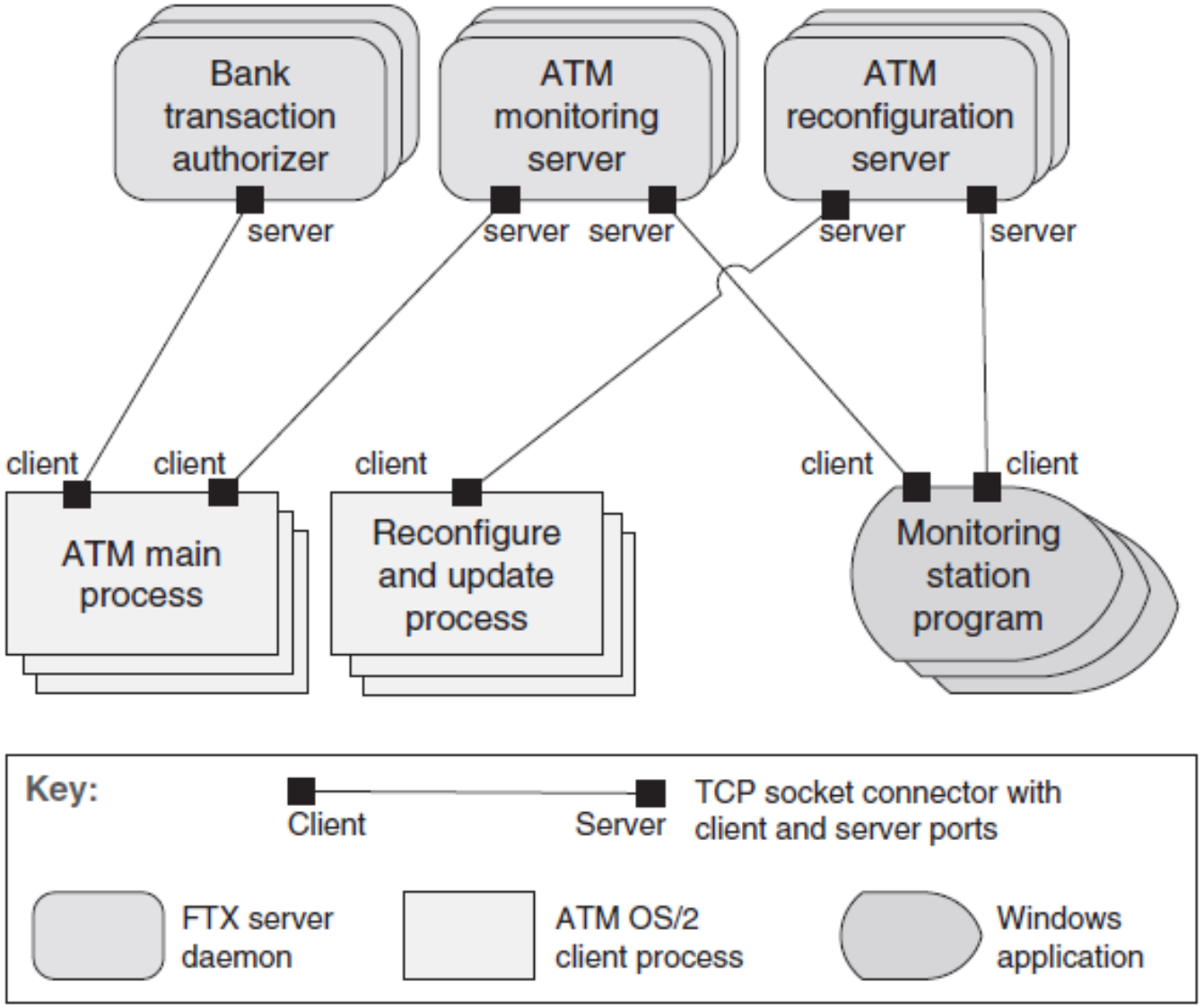

FIGURE 13.9 The client-server architecture of an ATM banking system

Client-Server Solution - 1

- Overview: Clients initiate interactions with servers, invoking services as needed from those servers and waiting for the results of those requests.
- **Elements**:
  - *Client*, a component that invokes services of a server component. Clients have ports that describe the services they require.
  - *Server:* a component that provides services to clients. Servers have ports that describe the services they provide.
  - *Request/reply connector:* a data connector employing a request/reply protocol, used by a client to invoke services on a server.
    - Important characteristics include whether the calls are local or remote, and whether data is encrypted.

Client-Server Solution- 2

- **Relations**: The *attachment* relation associates clients with servers.

- Constraints:
  - Clients are connected to servers through request/reply connectors.
  - Server components can be clients to other servers.

- Weaknesses:
  - Server can be a performance bottleneck.
  - Server can be a single point of failure.
  - Decisions about where to locate functionality (in the client or in the server) are often complex and costly to change after a system has been built.

Finer Points of Client-Server

- The computational flow of pure client-server systems is asymmetric:
  - clients initiate interactions by invoking services of servers. Thus, the client must know the identity of a service to invoke it, and clients initiate all interactions.
  - In contrast, servers do not know the identity of clients in advance of a service request and must respond to the initiated client requests.

- The client-server pattern separates client applications from the services they use.
  - This pattern simplifies systems by factoring out common services, which are reusable.
  - Because servers can be accessed by any number of clients, it is easy to add new clients to a system.

Usages of Client-Server

- Some common examples of systems that use the client-server pattern are these:
  - Information systems running on local networks where the clients are GUI launched applications and the server is a database management system.
  - Web-based applications where the clients are web browsers and the servers are components running on an e-commerce site

- The World Wide Web is the best-known example of a system that is based on the client-server pattern, allowing clients (web browsers) to access information from servers across the Internet using HyperText Transfer Protocol (HTTP).